HERMES · INFRASTRUCTURE REBUILD PROPOSAL

Scaling SMS to 60 MPS

A modular, queue-driven rebuild of the Hermes send pipeline and dashboard.

Prepared for Carson Craig
Date May 19, 2026
Target 20 MPS × 3 numbers

60 MPS
Sustained aggregate throughput goal (20×3 toll-free numbers)

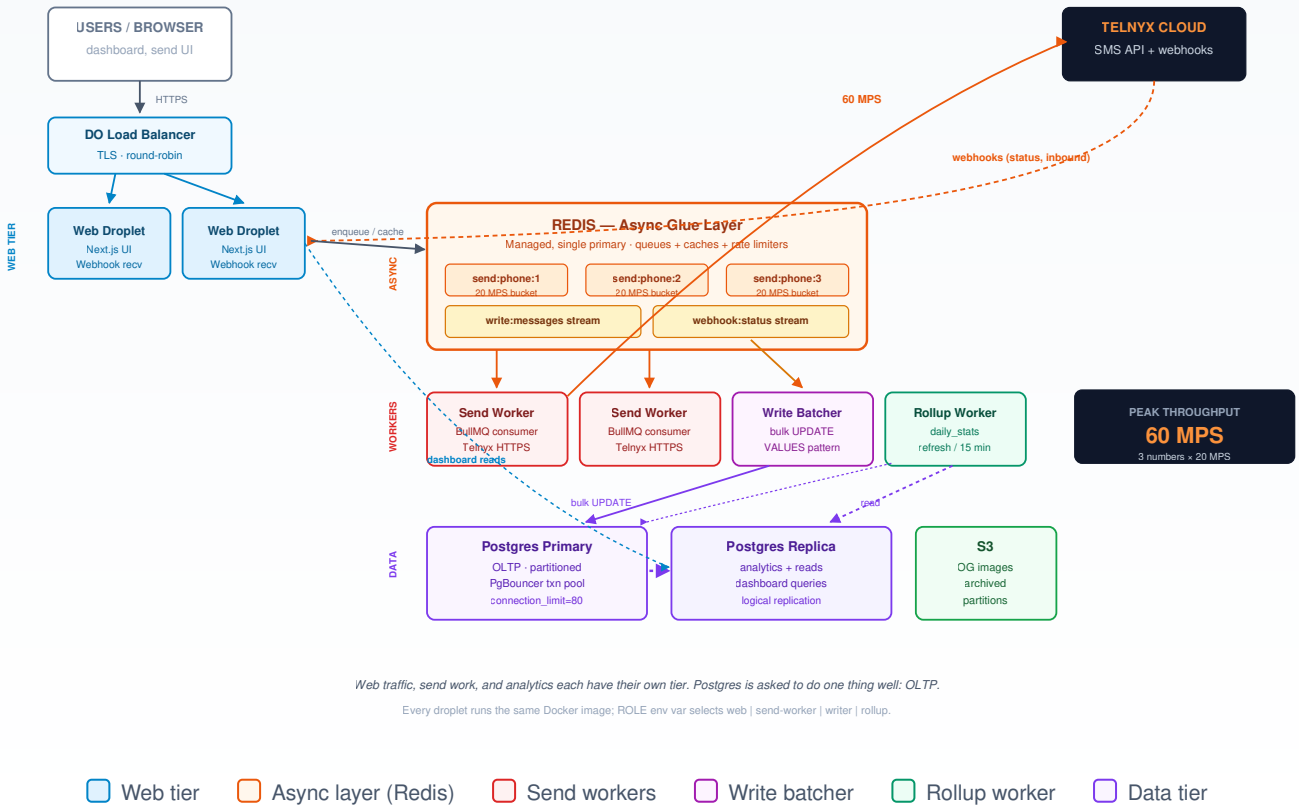
~6×
Throughput uplift vs. estimated current ceiling (~5–10 MPS)

<300ms
Target dashboard first paint (vs. multi-second today)

~$190/mo
All-in DigitalOcean infra (droplets + managed services)

The send loop, the dashboard, and every Telnyx webhook all run inside the same Next.js process, against the same 20-connection Postgres pool. **The bottleneck is architecture, not code.** Per-message DB cost and per-number rate budgets are symptoms; the fix is to separate concerns onto purpose-built workers with Redis between them.

1 Target Architecture



Web traffic, send work, and analytics each have their own tier. Postgres is asked to do one thing well: OLTP.

Every droplet runs the same Docker image; ROLE env var selects web | send-worker | writer | rollup.

2 Why It Can't Hit 60 MPS Today — and What the Rebuild Changes

CURRENT SHORTCOMINGS

- **One process does everything.** Sends, webhooks, dashboard, and cron all share one Next.js event loop and one Postgres pool (currently 20 connections).
- **Per-message DB overhead is unavoidable.** Each successful send takes 2 writes (queued → sent) plus another on webhook. A 100k blast = ~300+ MB of JSON churn just on `telnyxMetadata`.
- **"Per-number rate limit" is a label, not a budget.** Within a batch, every message for a number fires in parallel with no semaphore.
- **Interactive `$transaction` per group** serializes N UPDATEs while holding a pool connection.
- **Dashboard does OLAP on OLTP.** 11+ unindexed aggregates per load, no Redis, no rollup table, conversations re-derived in JS.
- **Immediate jobs aren't recoverable.** If the dyno restarts mid-send, the job stays stuck in `processing` forever.

WHAT THE REBUILD DOES

- **Send loop leaves the web process.** Dedicated BullMQ worker droplets pull from per-number Redis queues with real token-bucket rate limits.
- **Writes are batched centrally.** One write-batcher worker drains a Redis stream and applies `UPDATE… FROM VALUES` in one round trip per ~200 messages.
- **Webhooks return in <10ms.** Receivers push to a Redis stream; the same write-batcher applies them. Webhook bursts no longer touch the sender's pool.
- **Postgres splits in two.** Primary handles OLTP only. Read replica + Redis caches absorb every dashboard / analytics read.
- **A `Conversation` table + `MessageDailyStats` rollup** replace JS grouping and on-demand aggregation.
- **One Docker image, ROLE env var.** Standardized droplets behind a DO Load Balancer. Scale by adding droplets, not by rewriting.

3 Before vs. After

DIMENSION	TODAY	POST-REBUILD	IMPACT
Sustained throughput	~5–10 MPS aggregate before pool exhaustion	60 MPS (20 × 3 numbers); horizontal headroom beyond	**~6–12×**
DB ops per outbound message	~1.4 round trips amortized (worse during webhook flood)	~0.015 round trips amortized via batched VALUES updates	**~100×**
Per-number rate limit	Inter-batch sleep; intra-batch unbounded	Redis token bucket per queue, enforced before each Telnyx call	**real**
Dashboard p95 load	Seconds — 11+ unindexed aggregates per page	<300ms via replica + Redis caches + rollup table	**10×+**
Webhook handling	Per-webhook DB round trip on sender's pool	Push to Redis stream, return 200; bulk-applied async	**~10ms**
Failure recovery	Immediate jobs stuck in `processing` forever on dyno restart	BullMQ retries + DLQ; idempotency key on every Telnyx call	**safe**
Infra primitive	1 App Platform app + 1 Postgres	Standardized droplets behind LB + managed Redis + Postgres primary + replica	**modular**

DIMENSION	TODAY	POST-REBUILD	IMPACT
Monthly cost	~$60–120 (App Platform + DB)	~$190 all-in (2 web + 2 send + writer + rollup + LB + Redis + DB × 2)	**marginal**

4 Phased Rollout — ~7–9 Weeks

PHASE 0

Quick wins on current infra

~1 week

Bump pool to 80, replace per-group transaction with raw VALUES UPDATE, stop double-writing `telnyxMetadata`, fix immediate-job recovery, add Telnyx idempotency keys. Should buy ~15–30 MPS without architectural change.

PHASE 1

Read replica + Redis + dashboard caches

~2 weeks

Provision DO Managed Redis + Postgres replica. Add Conversation table + daily-stats rollup worker. Route dashboard / analytics / list pages to replica with Redis caching. Dashboard goes from seconds to <300ms.

PHASE 2

BullMQ queues + worker tier

~2–3 weeks

Build send-worker, write-batcher, webhook-processor processes. Refactor `executeDedicatedSend` into a queue submitter. Feature-flag legacy vs. queue. 60 MPS achieved behind a flag for safe rollout.

PHASE 3+4

Droplet migration + polish

~2–3 weeks

Unified Docker image with ROLE env var. Migrate DNS to droplets behind DO LB. Partition `messages` by month. Replace remaining ILIKE searches with trigram indexes. Decommission App Platform.

5 Design Principles

- **One job per process.** Web droplets serve UI and accept webhooks. Send droplets call Telnyx. Writer droplets touch the DB. Mixing them is what got us here.
- **Postgres is for OLTP. Redis is for in-flight state. The replica is for reading.** Never ask Postgres to be a queue or an analytics warehouse on the hot path.
- **Idempotency everywhere.** Idempotency key on every Telnyx call, unique DB constraint on `(campaignId, contactId, direction)`, idempotent webhook upserts. Retries never double-send.
- **Standardize the unit of scale.** One Docker image, one droplet size, one bash command to bring up a new worker. No bespoke infra per role.
- **Observability is not optional.** MPS-per-number, queue depth, batcher lag, pool utilization, dashboard p95 — all charted before scaling traffic. You can't tune what you can't see.

Full technical detail in `HERMES_REBUILD_PROPOSAL.md` · Hermes · May 2026